

# VIDEOJUEGOS PARA CONSOLAS / PC

PRACTICA Streaming de niveles

Carlos Plaza Cuenca | 2024 - 2025  
GITAV - S2

[carlos.plazacuenca@ceu.es](mailto:carlos.plazacuenca@ceu.es)



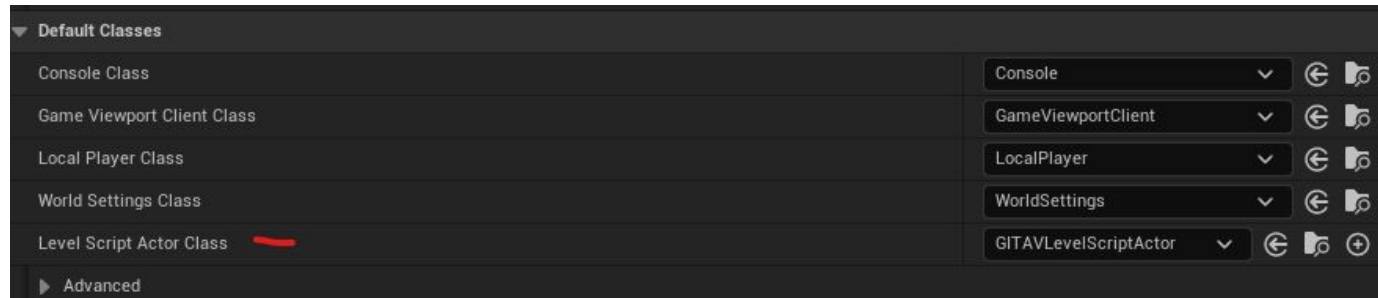


# Practica - Streaming de niveles

## Creando un gestor de carga/descarga de niveles

### OPCION 1 - LevelScriptActor

- Creamos una nueva clase C++, derivada de LevelScriptActor
- En el editor, marcamos como Level Manager nuestra clase. Vamos a Project Settings -> Engine - General Settings -> Level Script Actor Class





# Practica - Streaming de niveles

## Creando un gestor de carga/descarga de niveles

### OPCION 1 - LevelScriptActor

- Ahora en Visual Studio completaremos el código. Haremos una función LoadTargetLevel para cargar un nuevo nivel mediante el método de Streaming.

Esta función debe ser de tipo BlueprintCallable y recibirá tres parámetros.

```
void AGITAVLevelScriptActor::LoadTargetLevel(const FName LevelName, bool bShouldBeLoaded, bool bShouldBeVisible)
{
    UGameplayStatics::LoadStreamLevel(this, LevelName, bShouldBeVisible, true, FLatentActionInfo());
}
```



# Practica - Streaming de niveles

## Creando un gestor de carga/descarga de niveles

### OPCION 1 - LevelScriptActor

- De esta forma tendremos nuestro propio nodo que podremos invocar en cualquier Level Blueprint





# Practica - Streaming de niveles

## Creando un gestor de carga/descarga de niveles

### OPCION 1A - LevelScriptActor con Callbacks

- Para tener más control, vamos a añadir la opción de recoger un callback para poder actuar en consecuencia cuando la carga haya terminado.
- Para ello, el parámetro LatentInfo debemos crearlo nosotros y personalizarlo.

```
void AGITAVLevelScriptActor::LoadTargetLevel(const FName LevelName, bool bShouldBeLoaded, bool bShouldBeVisible)
{
    FLatentActionInfo info;
    info.CallbackTarget = this;
    info.ExecutionFunction = "OnLevelLoaded";
    info.Linkage = 0;
    info.UUID = 123;
    UGameplayStatics::LoadStreamLevel(this, LevelName, bShouldBeVisible, true, FLatentActionInfo());
}

void AGITAVLevelScriptActor::OnLevelLoaded()
{
    UE_LOG(LogTemp, Warning, TEXT("LEVEL FINISHED LOADING"));
}
```



# Practica - Streaming de niveles

## Creando un gestor de carga/descarga de niveles

### OPCIÓN 2 - Tarea asíncrona

- En este caso, no haremos uso del LevelScriptActor sino que crearemos una tarea asíncrona. Para ello crearemos una nueva clase C++ derivada de **BlueprintAsyncActionBase**.
- En lugar de callbacks, haremos uso de Delegates para notificar cuando ha terminado la carga del nivel.

```
UCLASS()  
class PRACTICA5_API UAsyncTaskStreamSublevels : public UBlueprintAsyncActionBase  
{  
    GENERATED_BODY()  
  
    DECLARE_DYNAMIC_MULTICAST_DELEGATE(FFROnCompleteLoadDelegate);  
}
```



# Practica - Streaming de niveles

## Creando un gestor de carga/descarga de niveles

### OPCIÓN 2 - Tarea asíncrona

- El concepto de esta forma de cargar niveles es diferente. En este caso la propia clase devuelve una instancia de sí misma. Necesitamos utilizar una función “static”
- La función static puede tener tantos parámetros como quieras, y debe utilizarse para inicializar la instancia de la tarea.
- La propia lógica de la tarea se dispara sobrescribiendo la función “Activate()”

```
UAsyncTaskStreamSublevels* UAsyncTaskStreamSublevels::StreamSublevel(UObject* WorldContextObject, const FName SublevelToLoad, bool bVisibleAfterLoad)
{
    UAsyncTaskStreamSublevels* StreamSublevels = NewObject<UAsyncTaskStreamSublevels>();

    StreamSublevels->WorldReference = WorldContextObject;
    StreamSublevels->SublevelToLoad = SublevelToLoad;
    StreamSublevels->bShouldBeVisible = bVisibleAfterLoad;

    return StreamSublevels;
}
```